# Guía del alumno

## Módulo 2, Manipulación de datos

Ruta guiada

Análisis Avanzado de Datos con Python

### Cómo usar esta guía

Esta guía acompaña al Lab 02 guiado y sigue sus dos partes en el mismo orden y con los mismos nombres del cuaderno. La Parte 1, En clase, tiene cinco pasos, de Paso 1 de 5 a Paso 5 de 5, y se hace en clase. La Parte 2, Para estudiar, tiene cuatro estudios, de Estudio 1 de 4 a Estudio 4 de 4, y se hace después de clase con esta guía, una vez corrida la Parte 1. Cada paso trae el código y la respuesta tal como aparecen en Colab, y una explicación en palabras simples.

No hay que escribir código. Lo que cambia se cambia en las listas desplegables, y después se vuelve a correr la celda. El lab responde tres preguntas. Cómo se eligen las filas que interesan, cómo se resume una tabla grande en pocas cifras y cómo se calcula una columna nueva para toda la tabla.

### Lo que vas a poder hacer al terminar

- Filtrar una tabla con dos condiciones a la vez.
- Crear una columna nueva calculada para todas las filas.
- Resumir una tabla por grupos, con uno o varios cálculos.
- Armar una tabla de doble entrada, como una tabla dinámica.
- Desconfiar de una cifra global y mirarla también por grupo.

Todas las respuestas de esta guía vienen de una corrida del cuaderno. Los datos del curso son sintéticos, inventados para aprender, y no se citan como datos del sector.

# Parte 1 · En clase

Cinco pasos, uno detrás del otro, que se hacen en clase en el mismo orden del cuaderno.

## Paso 1 de 5 · ¿Con qué datos vamos a trabajar?

La Parte 1 trabaja con dos tablas del curso, las centrales y la generación de cada hora de 2024, unidas en una sola.

### Paso 1 de 5

```
# Paso 1
DATOS = "https://raw.githubusercontent.com/RodrigoMSB/curso-python-minergia/main/datos/"
import pandas as pd

centrales = pd.read_csv(DATOS + "centrales.csv")
generacion = pd.read_csv(DATOS + "generacion.csv")

# La generación de cada hora con la ficha de su central al lado, como en el Lab 01.
gen = pd.merge(generacion, centrales, on="central", how="left")
print("Filas de la tabla unida", len(gen))
gen.head()
```

```
Filas de la tabla unida 175680
        fecha  hora                central     mwh tecnologia  region  \
0  2024-01-01     0  Central Río Manso Alto  163.69      hidro  Biobío
1  2024-01-01     1  Central Río Manso Alto  179.81      hidro  Biobío
2  2024-01-01     2  Central Río Manso Alto  173.77      hidro  Biobío
3  2024-01-01     3  Central Río Manso Alto  170.39      hidro  Biobío
4  2024-01-01     4  Central Río Manso Alto  169.84      hidro  Biobío

   potencia_mw  anio_inicio
0          420         2004
1          420         2004
2          420         2004
3          420         2004
4          420         2004
```

La celda lee las dos tablas desde la carpeta de datos del curso, igual que en el Lab 01. Después une la generación con la ficha de cada central, con `left`, para que cada hora de generación traiga al lado la tecnología, la región y la potencia de su central. Esa tabla unida se llama `gen`.

`gen` tiene 175680 filas, las mismas de la generación. Contar las filas después de unir es la regla del Lab 01, y aquí confirma que la unión no perdió ni duplicó nada. `head` muestra las primeras cinco filas.

La tabla es más ancha que la hoja, así que en esta guía las últimas columnas aparecen más abajo. En Colab se ve completa.

## Paso 2 de 5 · Tu turno, filtra con dos condiciones

Un filtro puede pedir dos cosas a la vez. Acá, una tecnología en una región.

### Paso 2 de 5

```
# Paso 2
tecnologia = "solar" #@param ["hidro", "solar", "eolica", "gas", "carbon", "diesel"]
region = "Antofagasta" #@param ["Antofagasta", "Atacama", "Coquimbo", "Valparaíso",
↪  "Metropolitana", "Biobío", "Los Lagos"]

# El signo & se lee como "y". Cada condición va entre paréntesis.
elegidas = gen[(gen["tecnologia"] == tecnologia) & (gen["region"] == region)]

print("Filas encontradas", len(elegidas))
total = round(elegidas["mwh"].sum())
print(f"✓ Generación de {tecnologia} en {region}", total, "MWh")
```

```
Filas encontradas 26352
✓ Generación de solar en Antofagasta 479681 MWh
```

La tecnología y la región se eligen en las dos listas desplegables. El signo `&` se lee como *y*, así que la celda se queda con las filas que cumplen las dos condiciones al mismo tiempo. Cada condición va entre paréntesis.

Con solar en Antofagasta hay 26352 filas, que son las horas del año de las solares de esa región, y entre todas generaron 479681 MWh en 2024. Elige otra combinación y vuelve a correr la celda. Si la tecnología no existe en esa región, la celda encuentra cero filas.

## Paso 3 de 5 · Tu turno, ¿cuánto generó cada grupo en el año?

Agrupar es lo mismo que hace una tabla dinámica. Junta las filas de cada grupo y suma su generación.

### Paso 3 de 5

```
# Paso 3
# Agrupar es lo mismo que hace una tabla dinámica.
# Junta las filas de cada grupo y suma su generación.
agrupar_por = "tecnologia" #@param ["tecnologia", "region"]

por_grupo = gen.groupby(agrupar_por)["mwh"].sum().round()
por_grupo.sort_values(ascending=False)
```

```
tecnologia
hidro     6106351.0
gas       5094126.0
carbon    4666823.0
eolica    1565899.0
solar      671677.0
diesel      40710.0
Name: mwh, dtype: float64
```

La lista desplegable elige cómo agrupar, por tecnología o por región. `groupby` junta las filas que tienen el mismo valor y `sum` suma los MWh de cada grupo. El resultado se redondea y se ordena de mayor a menor.

Por tecnología, la hidro es la que más generó en 2024, con 6106351 MWh. La solar queda quinta con 671677, aunque es segunda en potencia instalada, porque de noche no genera.

Elige región y vuelve a correr la celda. Biobío es la región que más generó, con 8539297 MWh. La última línea de la salida, la que dice `Name` y `dtype`, la agrega Python para describir el resultado y se puede ignorar.

> **POTENCIA INSTALADA NO ES ENERGÍA GENERADA**
>
> En este parque inventado la solar es segunda en potencia instalada y quinta en energía generada, porque de noche no genera.

## Paso 4 de 5 · ¿Cuánto generó cada región con cada tecnología?

Una tabla de doble entrada cruza dos cosas. Acá, las regiones en las filas y las tecnologías en las columnas.

### Paso 4 de 5

```
# Paso 4
# pivot_table arma una tabla de doble entrada, como una tabla dinámica.
# fill_value=0 pone cero donde una región no tiene esa tecnología.
cruce = gen.pivot_table(index="region", columns="tecnologia",
                        values="mwh", aggfunc="sum", fill_value=0)
cruce.round()
```

```
tecnologia        carbon   diesel     eolica        gas      hidro     solar
region
Antofagasta    1967910.0      0.0        0.0        0.0        0.0  479681.0
Atacama              0.0      0.0        0.0        0.0        0.0  191996.0
Biobío         2698913.0      0.0        0.0  1688455.0  4151930.0       0.0
Coquimbo             0.0      0.0  1332263.0        0.0        0.0       0.0
Los Lagos            0.0      0.0        0.0        0.0  1954422.0       0.0
Metropolitana        0.0  40710.0        0.0  1222999.0        0.0       0.0
Valparaíso           0.0      0.0   233637.0  2182673.0        0.0       0.0
```

`pivot_table` arma la tabla de doble entrada, como una tabla dinámica con filas y columnas. Cada casilla es la generación de 2024 de esa tecnología en esa región.

Donde una región no tiene esa tecnología aparece un cero, porque la celda lo pide con `fill_value=0`. La tabla deja ver de un vistazo qué genera cada región. Biobío, por ejemplo, tiene carbón, gas e hidro.

## Paso 5 de 5 · ¿Qué tan seguido genera cada tecnología?

El **factor de planta** es lo que generó una central dividido por lo máximo que podía generar. Dice qué parte del tiempo la central produce como si estuviera a plena potencia.

### Paso 5 de 5

```
# Paso 5
# Una columna nueva calculada para las 175.680 filas de una vez.
gen["factor_planta"] = gen["mwh"] / gen["potencia_mw"]

factor = gen.groupby("tecnologia")["factor_planta"].mean().round(3)
factor.sort_values(ascending=False)
```

```
tecnologia
gas       0.663
carbon    0.640
hidro     0.549
eolica    0.315
diesel    0.103
solar     0.080
Name: factor_planta, dtype: float64
```

La celda crea la columna `factor_planta` para las 175680 filas de una vez, dividiendo la generación de cada hora por la potencia de la central. Después saca el promedio por tecnología.

El gas tiene el factor de planta más alto, 0.663, y la solar el más bajo, 0.080, porque de noche no genera.

> **ESTAS CIFRAS SON INVENTADAS**
>
> Los datos del curso son sintéticos. Sirven para aprender la técnica, pero no describen el sistema eléctrico real y no se citan en un informe.

# Parte 2 · Para estudiar

Estos pasos se hacen después de clase, con esta guía. Antes de empezar hay que correr toda la Parte 1 del cuaderno, porque los estudios usan sus tablas.

## Estudio 1 de 4 · ¿Se consume menos el fin de semana?

La demanda cambia según el día. Esta celda compara los días de semana con el sábado y el domingo.

### Estudio 1 de 4

```
# Estudio 1
demanda = pd.read_csv(DATOS + "demanda.csv", parse_dates=["fecha"])

# Una columna nueva que dice si cada fila cae en sábado o domingo.
# dayofweek numera los días de 0, lunes, a 6, domingo.
demanda["es_finde"] = demanda["fecha"].dt.dayofweek >= 5

promedio = demanda.groupby("es_finde")["mwh"].mean()
print(promedio.round(1))

caida = 100 * (1 - promedio[True] / promedio[False])
print(f"✓ El fin de semana se consume un {caida:.1f} por ciento menos")
```

```
es_finde
False    231.2
True     213.1
Name: mwh, dtype: float64
✓ El fin de semana se consume un 7.8 por ciento menos
```

La celda lee la demanda de cada hora en cada región, con `parse_dates` para que la fecha quede como fecha. Después crea una columna nueva, `es_finde`, que dice para cada fila si cae en sábado o domingo. Python numera los días de la semana de 0, el lunes, a 6, el domingo, así que el fin de semana son los días 5 y 6.

Con esa columna saca el promedio de la demanda en cada grupo. En días de semana el promedio por región y por hora es 231.2 MWh, y el fin de semana 213.1. El fin de semana se consume un 7.8 por ciento menos.

## Estudio 2 de 4 · Tu turno, resume con varios cálculos

Un resumen puede traer varios cálculos a la vez, cada uno en su columna.

### Estudio 2 de 4

```
# Estudio 2
agrupar_por = "tecnologia" #@param ["tecnologia", "region"]

# Cada línea es una columna del resumen. A la izquierda el nombre,
# a la derecha la columna y el cálculo.
resumen = gen.groupby(agrupar_por).agg(
    total_mwh=("mwh", "sum"),
    promedio_hora=("mwh", "mean"),
    centrales=("central", "nunique"),
)
resumen.round(1).sort_values("total_mwh", ascending=False)
```

```
            total_mwh  promedio_hora  centrales
tecnologia
hidro       6106351.4          139.0          5
gas         5094126.1          193.3          3
carbon      4666823.0          265.6          2
eolica      1565899.4           44.6          4
solar        671677.2           15.3          5
diesel         40709.5            4.6          1
```

La lista desplegable elige cómo agrupar, por tecnología o por región. Cada línea dentro de `agg` es una columna del resumen. A la izquierda va el nombre de la columna y a la derecha la columna de origen y el cálculo.

`total_mwh` suma la generación, `promedio_hora` saca el promedio por hora y `centrales` cuenta cuántas centrales distintas hay en el grupo. El conteo ayuda a leer los otros números. La hidro suma más que el carbón, pero tiene 5 centrales contra 2.

Elige región y vuelve a correr la celda.

## Estudio 3 de 4 · ¿Sube el precio cuando sube la demanda?

Esta pregunta junta dos tablas, los precios y la demanda, para ver si se mueven juntos.

### Estudio 3 de 4

```
# Estudio 3
# Los precios vienen de un servicio web, como en el Estudio 4 del Lab 01.
import requests
respuesta = requests.get(DATOS + "precios_nudo.json").json()
precios = pd.DataFrame(respuesta["datos"])

# La demanda se lee sin parse_dates. Así su fecha queda como texto,
# igual que la de los precios, y las dos columnas calzan.
demanda = pd.read_csv(DATOS + "demanda.csv")

# Se unen por fecha, hora y región a la vez.
mix = pd.merge(precios, demanda, on=["fecha", "hora", "region"], how="inner")
print("Filas unidas", len(mix))

# La correlación va de -1 a 1. Cerca de 1 es que suben juntos.
correlacion_total = round(mix["precio_usd_mwh"].corr(mix["mwh"]), 3)
print("✓ Correlación con todas las regiones", correlacion_total)
```

```
Filas unidas 5208
✓ Correlación con todas las regiones 0.979
```

La celda lee los precios desde el servicio web del curso, como en el Estudio 4 del Lab 01, y lee la demanda sin `parse_dates`. Así la fecha queda como texto en las dos tablas y se puede comparar. Después las une por fecha, hora y región a la vez. Quedan 5208 filas unidas, que son las horas de enero en las siete regiones.

La **correlación** mide si dos columnas suben y bajan juntas. Va de menos uno a uno. Cerca de uno quiere decir que cuando una sube, la otra también. Con todas las regiones juntas da 0.979, casi perfecta. El estudio siguiente muestra por qué ese número engaña.

## Estudio 4 de 4 · ¿Y dentro de cada región?

La misma correlación, ahora mirando una región a la vez.

### Estudio 4 de 4

```
# Estudio 4
region = "Metropolitana" #@param ["Antofagasta", "Atacama", "Coquimbo", "Valparaíso",
↪  "Metropolitana", "Biobío", "Los Lagos"]

una = mix[mix["region"] == region]
correlacion = round(una["precio_usd_mwh"].corr(una["mwh"]), 3)
print(f"Correlación dentro de {region}", correlacion)
print("Correlación con todas las regiones", correlacion_total)
```

```
Correlación dentro de Metropolitana 0.917
Correlación con todas las regiones 0.979
```

La región se elige en la lista desplegable. La celda deja solo las filas de esa región y calcula la correlación entre precio y demanda dentro de ella. Con Metropolitana da 0.917, menos que el 0.979 de todas las regiones juntas.

Elige otras regiones y vuelve a correr la celda. En algunas la correlación es mucho más baja.

> **UN TOTAL PUEDE ENGAÑAR**
>
> Con todas las regiones juntas la correlación es 0,979, casi perfecta. Dentro de cada región va de 0,125 a 0,917. La relación fuerte aparece porque las regiones grandes tienen a la vez más demanda y precios más altos. Antes de informar una cifra global, se mira por grupo.

# Para repasar

## Referencia rápida

| Lo que se escribe | Para qué sirve |
|---|---|
| `(cond1) & (cond2)` | Las filas que cumplen las dos condiciones |
| `tabla["nueva"] = ...` | Crea una columna nueva para todas las filas |
| `groupby("col")["mwh"].sum()` | Suma por grupo |
| `groupby(...).mean()` | Promedio por grupo |
| `groupby(...).agg(...)` | Varios cálculos por grupo, uno por columna |
| `pivot_table(index=..., columns=...)` | Tabla de doble entrada |
| `sort_values(ascending=False)` | Ordena de mayor a menor |
| `a.corr(b)` | Correlación entre dos columnas, de menos uno a uno |

## Si algo sale mal

| Lo que aparece en pantalla | Qué pasó |
|---|---|
| `NameError: name 'gen' is not defined` | Corriste un paso sin correr antes el Paso 1, que es el que lee y une las tablas. |
| `NameError: name 'mix' is not defined` | El Estudio 4 usa la tabla que arma el Estudio 3. Corre el Estudio 3 antes. |
| El Paso 2 encuentra cero filas | No es un error. Esa tecnología no tiene centrales en esa región. |
| La respuesta no cambió después de cambiar la lista | Cambiar la lista no corre la celda. Hay que volver a apretar el botón de play. |

## Tres preguntas de repaso

### 1. Qué quiere decir el signo & en un filtro

Que la fila tiene que cumplir las dos condiciones a la vez. Se lee como y.

### 2. Para qué sirve contar las centrales en un resumen

Para leer bien los totales. Un grupo puede sumar más solo porque tiene más centrales.

### 3. Por qué la correlación de todas las regiones engaña

Porque junta regiones grandes, con más demanda y precios más altos, con regiones chicas. Dentro de cada región la relación es más débil.

# El Lab 02 guiado en una plana

Para imprimir y tener al lado durante el curso

## Los pasos de las dos partes

| N. | La pregunta | La idea |
|---|---|---|
| *Parte 1, en clase* | | |
| 1 | ¿Con qué datos vamos a trabajar? | Leer y unir dos tablas |
| 2 | Tu turno, filtra con dos condiciones | Un filtro con & |
| 3 | Tu turno, ¿cuánto generó cada grupo en el año? | Sumar por grupo |
| 4 | ¿Cuánto generó cada región con cada tecnología? | Tabla de doble entrada |
| 5 | ¿Qué tan seguido genera cada tecnología? | Una columna nueva para todas las filas |
| *Parte 2, para estudiar* | | |
| 1 | ¿Se consume menos el fin de semana? | Una columna nueva y un promedio |
| 2 | Tu turno, resume con varios cálculos | Varios cálculos por grupo |
| 3 | ¿Sube el precio cuando sube la demanda? | Unir por tres columnas y la correlación |
| 4 | ¿Y dentro de cada región? | La correlación por grupo |

## Lo que se lleva este lab

| | |
|---|---|
| Agrupar es una tabla dinámica | Junta las filas de cada grupo y calcula |
| Potencia no es energía | La solar es segunda en potencia y quinta en energía |
| Un resumen lleva su conteo | Así se leen bien los totales |
| Una cifra global se mira por grupo | El total puede esconder lo que pasa adentro |
| Las cifras del curso son inventadas | Sirven para aprender, no para un informe |

Las fechas en detalle, las tablas largas y anchas con melt y el trabajo con Polars están en la ruta completa de este módulo.